

Corrigé détaillé — Sujet d'entraînement B

Streams et Command/MVC

Exercice 1 — API Stream

Question 1 — nomsAdmis

```
import java.util.List;
import java.util.stream.Collectors;

public static List<String> nomsAdmis(List<Etudiant> etudiants) {
    return etudiants.stream()
        .filter(e -> e.getNote() >= 10)
        .map(Etudiant::getNom)
        .sorted()
        .collect(Collectors.toList());
}
```

La structure est le pipeline Stream le plus classique :

- `.stream()` ouvre le flux à partir de la liste ;
- `.filter(...)` est une opération **intermédiaire** qui ne garde que les étudiants vérifiant le prédicat (`getNote() >= 10`) ;
- `.map(Etudiant::getNom)` transforme chaque `Etudiant` restant en son nom (référence de méthode équivalente à `e -> e.getNom()`) ;
- `.sorted()` trie les `String` selon leur ordre naturel (alphabétique) ;
- `.collect(Collectors.toList())` est l'opération **terminale** qui matérialise le résultat en `List<String>` .

L'ordre des opérations compte : on filtre puis on transforme (on n'a pas besoin de trier les notes, seulement les noms), ce qui évite tout travail inutile.

Question 2 — moyenneGroupe

```
public static double moyenneGroupe(List<Etudiant> etudiants, String groupe) {
    return etudiants.stream()
        .filter(e -> e.getGroupe().equals(groupe))
        .mapToDouble(Etudiant::getNote)
        .average()
        .orElse(0);
}
```

`mapToDouble` transforme le `Stream<Etudiant>` en `DoubleStream` (un flux primitif spécialisé), ce qui permet d'appeler `.average()` directement — `average()` n'existe que sur les flux numériques spécialisés (`IntStream`, `LongStream`, `DoubleStream`), pas sur un `Stream<T>` générique. `average()` renvoie un `OptionalDouble` (car la moyenne d'un flux vide n'est pas définie), d'où le `.orElse(0)` final qui gère le cas demandé où aucun étudiant du groupe n'existe.

Question 3 — parGroupe

```
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public static Map<String, List<Etudiant>> parGroupe(List<Etudiant> etudiants) {
    return etudiants.stream()
        .collect(Collectors.groupingBy(Etudiant::getGroupe));
}
```

`Collectors.groupingBy` est un collecteur prêt à l'emploi qui construit directement une `Map<K, List<T>>` à partir d'une fonction de classification (ici `Etudiant::getGroupe`) : chaque étudiant est placé dans la liste correspondant à la clé renvoyée par cette fonction. C'est exactement le résultat demandé en une seule ligne, sans avoir à gérer soi-même la création des listes pour chaque nouvelle clé rencontrée.

Question 4 — meilleurEtudiant

```
import java.util.Comparator;
import java.util.Optional;

public static Optional<Etudiant> meilleurEtudiant(List<Etudiant> etudiants) {
    return etudiants.stream()
        .max(Comparator.comparingDouble(Etudiant::getNote));
}
```

`max` est une opération terminale qui attend un `Comparator<T>` et renvoie directement un `Optional<T>` (vide si le flux est vide, ce qui correspond exactement à la signature demandée). `Comparator.comparingDouble(Etudiant::getNote)` construit un comparateur qui ordonne les étudiants selon leur note, sans qu'on ait à écrire de classe de comparaison ni de boucle de recherche du maximum à la main.

Exercice 2 — Command et MVC

Question 1 — Interface Commande

```
public interface Commande {
    void executer();
}
```

C'est le cœur du pattern Command : on **encapsule une action** (et toutes les données nécessaires à son exécution) dans un objet qui expose une unique méthode d'exécution. Cela permet de manipuler des actions comme n'importe quel autre objet : les stocker dans une liste, les passer en paramètre, les rejouer, etc.

Question 2 — CommandPoserObjet

```

public class CommandePoserObjet implements Commande {

    private Salle salle;
    private ObjetZork objet;
    private Vue vue;

    public CommandePoserObjet(Salle salle, ObjetZork objet, Vue vue) {
        this.salle = salle;
        this.objet = objet;
        this.vue = vue;
    }

    public void executer() {
        salle.ajouterObjet(objet);
        vue.afficherMessage("Vous avez pose " + objet.getNom() + " dans la salle.");
    }
}

```

La commande retient toutes les références nécessaires (`salle` , `objet` , `vue`) au moment de sa **création**, et ne déclenche l'action qu'au moment où `executer()` est explicitement appelée — c'est ce découplage entre la création de l'action et son déclenchement qui constitue l'intérêt du pattern (on peut par exemple créer une commande à un endroit du code et ne l'exécuter que plus tard, ou plusieurs fois, ou la stocker pour un système d'historique/annulation).

Question 3 — Contrôleur

```

import java.util.ArrayList;
import java.util.List;

public class Controleur {

    private List<Commande> historique;

    public Controleur() {
        this.historique = new ArrayList<Commande>();
    }

    public void executer(Commande commande) {
        commande.executer();
        historique.add(commande);
    }
}

```

Le `Controleur` ne connaît que l'interface `Commande` — jamais les classes concrètes comme `CommandePoserObjet`. Cela signifie qu'on peut ajouter de nouveaux types de commandes (`CommandeRetirerObjet` , `CommandeDeplacer` , etc.) sans jamais modifier `Controleur` : c'est une application directe du principe ouvert/fermé (*open/closed principle*), rendu possible par le fait que `executer()` est polymorphe.

Question 4 — Pourquoi le Modèle ne doit pas appeler la Vue

Le principe respecté ici est la **séparation des responsabilités** propre à l'architecture MVC : le Modèle représente uniquement les données et la logique métier, indépendamment de toute façon de les afficher. S'il appelait directement `Vue.afficherMessage`, il deviendrait dépendant d'une représentation particulière (texte console) et impossible à réutiliser tel quel avec une autre vue (interface graphique, interface web, tests automatisés sans affichage...). C'est au **Contrôleur** — ou à la Commande elle-même, comme ici — de faire le lien entre l'action sur le Modèle et la mise à jour de la Vue, jamais au Modèle de le faire lui-même.

Pièges fréquents

1. **Mélanger Stream et boucle explicite** : utiliser un `for` pour parcourir le résultat d'un `.collect(...)` ou pour construire la `Map` à la main alors que `Collectors.groupingBy` le fait directement — cela ne respecte pas la consigne même si le résultat final est correct.
2. **Oublier qu' `average()` et consorts renvoient un type `Optional*`** (`OptionalDouble`, `OptionalInt` ...) et non directement un `double / int` — il faut systématiquement le débballer (`orElse`, `getAsDouble`, etc.).
3. **Faire exécuter la commande directement dans son constructeur** au lieu de dans `executer()` : cela casse tout l'intérêt du pattern (pouvoir différer, stocker, ou rejouer l'action) et empêche notamment la construction d'un historique cohérent dans `Contrôleur`.
4. **Faire afficher un message directement par `Salle`** (le Modèle) plutôt que de passer par la `Vue` via la Commande ou le Contrôleur — c'est exactement la violation de MVC que l'exercice cherche à éviter.